

Nika Blažica in Žan Grčar

Krpalnik (Patcher)

Seminarska naloga pri predmetu Sistemska programska oprema

MENTOR: doc. dr. Tomaž Dobravec

Povzetek

Seminarska naloga obravnava krpalnike (patcherje) kot ključen mehanizem za odpravljanje programskih napak in varnostnih ranljivosti v sodobnih informacijskih sistemih. Predstavljeni so osnovni pojmi, krpanje izvirne kode, binarno delta krpanje in krpanje med obratovanjem. Analizira tudi pristope različnih operacijskih sistemov k posodabljanju njihovih jeder in sistemskih komponent.

1. UVOD

V svetu računalništva obstaja nenehna potreba po rednem vzdrževanju in posodabljanju programske opreme, saj je ključnega pomena pri zagotavljanju varnosti in zanesljivosti delovanja sistemov, kot tudi aktualnosti v nenehno razvijajočem okolju. Popravki (patch) in krpalnik, ki jih namesti so ključen element sodobnega posodabljanja, saj omogočajo spremembe in rešitve brez potrebe po ponovni namestitvi programske opreme.

2. KRPALNIK

Vsako programsko opremo je treba občasno posodobiti. Posodobitve so najpogosteje potrebne zaradi napak v izvorni kodi (t. i. hroščev), ki lahko povzročijo nepravilno delovanje posameznih funkcionalnosti, ali zaradi varnostnih pomanjkljivosti. Takšne manjše spremembe imenujemo *popravek*, oziroma v angleščini *patch*. [6] [15]

Krpalnik ali *patcher* je programsko orodje, namenjeno avtomatizaciji nameščanja manjših popravkov programske opreme, pri čemer ni potrebno izvajati celotnega namestitvenega postopek znova. Krpalnik je praviloma del širšega sistema imenovanega *upravljalnik posodobitev* (*update manager*), ki preverja razpoložljivost novih popravkov in posodobitev programske opreme in jih namešča. Primer upravljalnika posodobitev je Windows Update na Windows operacijskem sistemu, ki samodejno išče posodobitve in jih namesti, ko se uporabnik za to odloči. [3]

2.1 Zgodovina

V računalništvu izhaja veliko izrazov iz fizičnih strojev, ki danes niso več v uporabi. Enako velja za angleški izraz patch (krpa). Ko je bila koda hranjena na preluknjanih trakih (punched tape), je bilo možno manjše spremembe prelepiti in nato narediti nove luknje. Trak je bil torej v resnici zakrpan. Za luknjane kartice, ki so nadomestile luknjan trak se je uporabljal podoben pristop. [5] [9] [13]

2.2 Funkcionalnosti in zgradba

Glavna naloga krpalnika je izvesti varen prehod iz starega stanja v novo posodobljeno stanje preko zaporedja preverjanj in operacij.

2.2.1 Branje popravka

Prvi korak v tem procesu je branje datoteke popravka, ki krpalniku, da navodila, kako kodo zakrpati. Preden pa se lahko loti krpanja, mora prepoznati trenutno različico programskega opreme, ter preveriti, ali je popravek sploh lahko nameščen na ta sistem. Poleg same različice sistema ali aplikacije mora preveriti tudi združljivost operacijskega sistema (arhitekture procesorja in nameščenih komponent) ter odvisnosti, ki morajo biti prisotne, da lahko popravek uspešno deluje. Popravki so namreč v večini primerov namenjeni točno določeni verziji strojne opreme. [17] [30]

Na podlagi teh informacij krpalnik odloči, katero strategijo posodobitve bo uporabil. Ali bo zamenjal celotne datoteke, uporabil diferencialni pristop ali posodobil posamezne komponente.

2.2.2 Validacija popravka

Po branju je potrebna validacija celotnega popravka. Krpalnik preveri integriteto datotek preko zgoščenih vrednosti, identifikatorjev ali česa podobnega. S tem preveri nedotaknjenost datotek iz zunanjih virov. [30]

2.2.3 Izvajalnik

Izvajalnik dejanko zakrpa originalno kodo po navodilih iz popravka, izvajati pa se mora po korakih. Najpogostejši način posodabljanja temelji na ustvarjanju kopije trenutne različice sistema, nato pa za vsako popravljeno datoteko ustvari začasno posodobljeno datoteko. Spet sledi preverjanje popravljenih datotek, s čimer se zagotovi, da med krpanjem ni prišlo do napake. Sledi atomarna menjava starih in novih začasnih datotek. Ob uspešnem zaključku se po potrebi izbriše kopija prejšnje različice. Če med postopkom pride do napake, se vse izveden spremembe ponastavijo in obnovi se prvotno stanje (rollback). [21] [30]

3. TEHNIKE KRPANJA

Ostaja več načinov krpanja kode, brez popolne ponovne namestitve.

3.1 Krpanje izvirne kode (source-code patching)

Pri tej vrsti krpanja imamo na voljo izvirno kodo, ki jo želimo posodobiti, ter tekstovno datoteko popravka, v kateri so zapisane razlike med originalno različico izvirne kode in novo različico. Ena glavnih značilnosti je, da je taka tekstovna datoteka berljiva. S pomočjo te datoteke se nova različica poustvari iz stare. [4]

3.1.1. Primerjava datotek

Celoten proces je torej dokaj preprost. Najprej moramo primerjati različici izvirne kode. Linux ima na primer ukaz `diff`, ki datoteke primerja vrstico za vrstico in proba ugotoviti najmanjšo množico potrebnih operacij, da iz ene datoteke dobimo drugo. Edine operacije, ki so mu na voljo so izbriši, spremeni in dodajaj. Algoritem temelji na *problemu najdaljšega skupnega podzaporedja* in tako pošče vrstice, ki se med različicama niso spremenile. Končni produkt je datoteka, ki vsebuje natančna navodila, kako iz enega fila zgraditi drugega. [8] [14].

Kako `diff` deluje, pogledjmo natančneje na primeru. Imamo dve datoteki:

```
# first.txt
Hello,
World
!

# second.txt
Hello,
Dearest
Universe
!!
```

`diff` vrstice v datoteki najprej oštevilči, v našem primeru od ena do štiri. Nato poišče najdaljše skupno podzaporedje, kar je v tem primeru prva vrstice: `Hello,`, vse po prvi vrstici pa se razlikuje. `first.txt` ima zdaj še dve vrstici, `second.txt` pa tri. Ker sta bili vrstici dva in tri

zamenjani z vrsticami od dva do štiri, `diff` to označi kot spremembo (change) in ne kot ločeno dodajanje ali odstranjevanje. Tako dobimo prvo vrstico rezultata: `2,3c2,4`, ki nam pove, da moramo vrstice od 2 do 3 v `first.txt` spremeniti v vrstice 2 fo 4 v `second.txt`. Kako jih moramo spremeniti, nam povedo naknadna navodila:

```
2,3c2,4
< World
< !
\ No newline at end of file
---
> Dearest
> Universe
> !!
\ No newline at end of file
```

Vrstice, ki se začnejo z `<` pridejo iz `first.txt`. To so vrstice, ki se bodo odstranile oziroma bodo nadomeščene. Sledi separator `---`, ki predstavlja ločnico med tem kaj je bilo v originalni datoteki (`first.txt`) in kaj bo v novi datoteki (`second.txt`). Vrstice, ki se začnejo z `>` pridejo iz `second.txt`. To so nadomestitvene vrstice. Vrstici `\ No newline at end of file` povesta, da se nobena od datotek ne konča z znakom `\n`. `diff` na to opozori, saj bi se morali vse POSIX (Portable Operating System Interface) datoteke končati s tem znakom. [10]

Ta rezultat je podan v *standardnem izhodnem formatu*. Obstajajo pa tudi mnogo drugih formatov, kot je *unified format* (ali *unidiff*). Ta format veliko projektov zahteva in je zato eden najbolj razširjenih formatov med razvijalci.

Git ima prav tako svoj ukaz `git diff`, ki prikaže razlike med delovnim drevesom in indeksom, med dvema drevesoma, spremembe, ki nastanejo zaradi združitve itd. Po privzetih nastavitvah git ustvari tudi datoteko popravka. Privzet rezultat je formata *unidiff* z dodanimi git metapodatki. Ti pokažejo morebitna preimenovanja, vsebuje informacije o indeksu in commitih in podpira barvanje izhoda. Poglejmo izhod, brez metapodatkov: [12]

```
@@ -1,3 +1,4 @@
Hello,
-World
-!
\ No newline at end of file
+Dearest
+Universe
+!!
```

V prvi vrstici je navedeno območje sprememb. Torej vrstice ena do tri v `first.txt` in ena do štiri v `second.txt`. – pomeni pred spremembo in + po spremembi. Nato so navedene vse spremembe. Vrstice, ki se začnejo s presledkom, ostanejo nespremenjene, vrstice, ki se začnejo z minusom, so odstranjene in vrstice, ki se začnejo s plus so dodane. Ponovno imamo tudi `\ No newline at end of file`, ki označuje, da se prejšnja vrstica ni končala z znakom `\n`. Tudi ukazi `read`, `commit`, `pull` in `merge` uporabljajo `git diff`, a je to uporabniku skrito.

[32]

3.1.2 Nameščanje popravka

Drugi korak je nameščanje popravka na staro izvirno kodo.

Linux ima zato ukaz `patch`, ki datoteko popravka namesti na originalno zbirno kodo. Ob zagonu `patch` razbere format datoteke popravka, razen če je format izrecno podan z uporabo opcij. Začetek in konec sporočila kot so glave in podobno poskusi preskočiti. Posodobljene različice ponavadi nadomestijo izvirnik, z uporabo opcije `-b` pa se ustvari tudi backup. Imena datotek, ki jih je treba zakrpati so ponavadi zapisana v datoteki popravka, če pa je datoteka samo ena, je to lahko storjeno tudi v ukazni vrstici. Če se del ne posodobi pravilno, se to zapiše v zavrtnitveni izhodni datoteki `.rej` kontekstnega ali unified formata. Številke vrstic v zavrtnitveni datoteki, so lahko drugačne od tistih v vhodni, saj predstavljajo približno lokacijo, kamor bi se zavrtnjen del kode moral zapisati v posodobljeni različici. [17]

V gitu lahko popravek namestimo z ukazom `git apply`, ki spremembe iz popravka namesti na trenutno delovno okolje (working directory). Z opcijo `--index`, se popravek namesti tudi na indeks, a to ne ustvari commita. [11]

3.1.3 Uporaba

Ta način krpanja je značilen za odprtokodne projekte, kjer je izvorna koda javno dostopna. Razvijalci lahko pripravijo različne vrste popravkov, kot so odprava programskih napak, podpora za določen jezik ali uvedba novih funkcionalnosti. Te spremembe se upravljalcem posredujejo v obliki popravkov, s čimer ni potrebno prenašati celotne izvirne kode, temveč le spremembe glede na obstoječo različico programske kode.

3.2 Binarno delta krpanje (binary patching)

Binarno krpanje je proces pri katerem krpamo že prevedeno, izvedljivo kodo, namesto zbirne kode. Za razliko od krpanja zbirne kode, ja ta vrsta krpanja bistveno bolj zahtevna, zato je tu preverjanje pravilnosti toliko bolj pomembno. Tudi tu obstaja na milijone različnih orodij, bistvo pa je pri vseh enako. Najprej je treba poiskati ustrezne dele kode in jih previdno popraviti, včasih pa je bolj smiselno nadomestiti kar celotno datoteko. [25]

Če nas zanimajo res samo razlike in nočemo nadomeščati celotnih datotek, govorimo o delta krpanju. Delta je razlika med dvema različicama, kar pomeni, da spet potrebujemo orodje diff. Binarne datoteke same po sebi niso berljive, zato ni smiselno, da bi bil izhod berljiv. Kljub temu je algoritem enak, saj lahko na binarne podatke in berljive podatke v obeh primerih gledamo kot na nize bajtov. [1]

Eden od algoritmov, ki se uporablja pri binarnem krpanju je *algoritem premika blokov*. Algoritem išče večje bloke podatkov, ki se pojavljajo v obeh različicah. Bloki, ki so skupni, se označijo kot kopirani, bloki, ki so edinstveni pa kot vstavljeni. Tak pristop omogoča učinkovitejše generiranje delta datotek, saj se vsak blok v delti zabeleži le enkrat, čeprav se nanj lahko sklicujemo večkrat. To bistveno zmanjša velikost delta datoteke, čeprav se s tem izgubi berljivost. [1]

Pri implementaciji algoritma premika blokov se pogosto uporabljajo *stiskalni ali hash algoritmi*, ki se izvedejo na obeh vhodnih datotekah, kar omogoči hitrejše prepoznavanje ujemajočih se bloke. Znane implementacije so xDelta in zDelta.

xDelta algoritem se osredotoča predvsem na hitrost in zato žrtvuje velikost končne delta datoteke. Ima linearno časovno in prostorsko zahtevnost in je zmožna generirati delto/patch formata VCDiff.

Po drugi strani se BSDiff osredotoča na kompaktnost delte in je optimiziran za izvršljive datoteke (executables). Časovna zahtevnost je $O((n+m)\log(n))$, kjer sta n in m velikosti vhodnih datotek, ter prostorsko zahtevnost $\max(17n, 9n+m) + O(1)$. Tudi ta algoritem ima svoj format. [1]

Eden glavnih problem binarnega delta krpanja je pomanjkanje standarda. Na trgu so velike količine orodij in knjižnic za primerjanje datotek, vsaka pa uporablja svoj format, ki ga posledično lahko uporablja samo proizvajalec popravka. Kljub temu, sta bila v zgodovina dva večja poskusa standardizacije formata popravkov/delt. [1]

Unix .patch format, ki ga je ustvaril GNU (GNU's not Unix), oziroma že omenjeni unidiff, je usmerjen v berljivost rezultat. Format se uporablja predvsem v orodjih za procesiranje besedil in orodjih za verzioniranje.

VCDiff je najbolj razširjen poskus ustvarjanje podatkovno in algoritmično neodvisnega formata. Razširil se je predvsem zaradi Googlovega SDCH (Shared Dictionary Compression for HTTP) projekta, ki je temeljil na formatu VCDiff. Danes je mnogo diff algoritmov zmožnih generirati delte/popravke v formatu VCDiff. [1]

Binarno delta krpanje se uporablja pri posodobitvah programske opreme, kje ni smiselno ali pa ni mogoče prenesti binarnih datotek. Delta posodabljanje se uporablja pri posodabljanju računalniških iger in mobilnih aplikacij, saj zmanjša velikost prenosa. Uporablja se tudi pri posodabljanju operacijskih sistemov, saj jedra ne moremo vsakič znova nameščati. Linux jedro (kernel) uporablja bspatch in xdelta za postopne posodobitve (incremental updates), Windows Update za svoje "express updates" uporablja delta-based patches. [23]

3.3 Krpanje med obratovanjem (Runtime patching, hot patching ali live patching)

Bistvo te tehnike je, da se nadgradnja zgodi, med tem ko sistem obratuje, ne da bi bilo potrebno sistem ustaviti ali ponovno zagnati. Je vrsta dinamičnega posodabljanja, ki posodablja kodo v spominu, namesto na disku. [19]

Uporablja se predvsem pri posodabljanju operacijskih sistemov, oziroma njihovih jeder, ter v nekaterih primerih za namestitev varnostnih popravkov brez ponovnega zagona. Uporablja se tudi pri posodabljanju oblakov in virtualk, saj močno zmanjša čas vzdrževanja. [19]

4. SISTEMSKO ODVISNA LOGIKA

4.1 Krpalnik jedra (kernela)

Krpanje jedra operacijskega sistema je seveda odvisno od operacijskega sistema, kljub temu pa je splošen postopek načeloma skupen.

Sistem prejme popravek, ki je bil prenesen ali pa dostavljen s strani storitve posodabljanja. Nato je preverjen kriptografski podpis in integriteta. Glede na popravek, se datoteke jedra spremenijo ali zamenjajo na disku, a še ne delujejo po novi različici. Šele ko je posodobljen zagonski nalagalnik jedra, se zazna nova različica jedra. Sistem se ponovno zažene in v spomin naloži popravljenno jedro v pomnilnik jedra. Tudi gonilniki, varnostni moduli in sistemske tabele se morajo ponovno inicializirati v novem jedru.

4.2 Windows

Postopek krpanja se pri Windowsu začne z Windows Update Orchestratorjem (WUO). WUO, skupaj z Windows Update Agentom (WUA) skrbi za pridobivanje popravkov, tako da periodično in naključno preišče Microsoft Update in pokliče vmesnike za namensko programiranje (API), kot je WSUS (Windows Server Update Services). Vse popravke nato primerja s trenutnim sistemom, da ugotovi, ali je popravek namenjen našemu računalniku (različica Windowsa, zadnji popravki, ipd.). Uporablja LCU (Latest cumulative Update) format za krpanje oz. posodabljanje sistema, ki hrani vse popravke in posodobitve za to verzijo sistema, in omogoča, da se vsi naložijo hkrati.

Popravke in različice lahko naložimo tudi sami z Windows Update Catalog, iz kjer naložimo `.msu` datoteko, ter nato poženemo Windows Update Standalone Installer (WUSA), ki preveri `.msu` datoteko ter prenese naložene popravke v zavarovan Windows Update sandbox. [20]

Windows ne uporablja neposredne kopiraj in zamenjaj logike, temveč Component-Based Servicing CBS, ki omogoča nadgrajevanje po komponentah, namesto po spremembah. CBS skrbi za celoten proces krpanja. Najprej pridobi popravek, ga razdeli na komponentah, glede na specifikacije in verzije ter jih po prioriteti razvrsti v servisno kopico (Service Stack). Nato pripravi novo verzijo komponente in jo potisne v WinSxS, trgovino, ki hrani vse Windows komponente.

Na vsakem koraku si zapiše, kaj je že spremenjeno in kaj še rabi spremeniti, da se lahko po potrebi naredi povratek v prejšnjo verzijo (rollback).

Ob zaključenem posodabljanju CBS potrdi vse spremembe tako, da zagonskemu nalagalniku pove, naj uporabi novo verzijo komponent, in označi, da je potreben ponovni zagon računalnik. [22]

Samostojno posodabljanje jedra, oz. krpanje s popravki iz neverificiranih virov, je v Windows operacijskem sistemu onemogočeno. [20]

4.2.1 Windows “Hot patching”

Windows omogoča tudi krpanje med obratovanjem. Prednost je, da za manjše popravke ponoven zagon računalnika ni potreben, da se popravek zažene. Sistem si že ob zagonu rezervira nekaj prostora v pomnilniku (HPAT - HotPATch), ki ga uporabi, ob pridobitvi popravka. Ko se funkcija popravlja prvič, ji HPAT stroj dodeli vnos, kamor zapiše trampolin kodo in ciljni naslov. Nato staro kodo prepíše v tabelo za možnost ponastavitve, in jo prepíše z novo kodo. Na koncu preusmeri izvajanje kode na star naslov. To omogoči neprekinjeno delovanje kode med popravljanjem. Seveda pa je tako krpanje omejeno, saj zahteva, da so binarne slike pripravljene tako, da je izvrševanje kode mogoče varno preusmeriti na začetku funkcije. [16] [19]

4.3 Linux

Linux, za razliko od Windowsa, nima ene glavne točke, ker lahko dostopa do posodobitve sistema. Posodobitve in krpanje izdaja vsak distributer posebej. Sistem najprej naloži metapodatke repozitorija distribucije, kot so seznam paketov, odvisnosti, ipd, nato pa jih preveri in potrdi. Avtentikacija sama je ponovno odvisna od distribucije. Pri večini distribucij se nova datoteka jedra namesti v `/boot` in namesti nove module v `/lib/modules/<verzija>`. Nato izdela oz. namesti nove *initramfs* (INIT RAM FileSystem), ki poskrbijo, da se ob ponovnem zagonu uporabijo pravilni gonilniki. Verzija se nato doda na zaganjalnik s sistemom GRUB (GRand Unified Bootloader) in bo ob ponovnem zagonu tudi izbrana. Za razliko od Windowsa, Linux starih različic sistema ne briše. [17] [18]

4.3.1 Linux “Live patching”

Krpanje med obratovanjem v linuxu poteka malo drugače, saj paket že pride od distributerja kot livepatch modul. Sistem tako ve, da gre za aktivno popravljanje. Ko se modul naloži, doda v podsistem preslikavo iz originalne funkcije v novo funkcijo. Jedro tu ne sme samo na pol priklopiti med funkcijami, če je katera od niti funkcijo ravno izvaja. Ta problem je rešen s postopnim preklapanjem. Ko so bile preslikave pripravljene, je jedro počasi preusmerjalo promet na nove funkcije iz starih in tako doseglo uspešen prehod na popravljeno verzijo.

4.4 Debian

Uradno Debian krpanje poteka po naslednjem procesu: Ko naloži podatke iz repozitorija, APT preveri podpisane metapodatke. Temelj zaupanja je, da računalnik ve, kaj je zaupljivo in kaj ne. Nato prenese pakete `.deb`, ter za vsakega pokliče `dpkg`. Ta vsak paket posebej najprej odpakira, nato pa namesti potrebne datoteke na pravilne sistemske lokacije, posodobi bazo nameščenih paketov ter pokliče skripte v paketu (hooks). Te novo različico registrirajo v `/boot`, posodobijo konfiguracijo GRUB, generirajo nove initramfs ter označijo jedro kot pripravljeno za ponovni zagon.

Če želi uporabnik ročno namestiti popravek, mora pravilno naložiti `.deb` pakete, ki jih dobi na uradni strani distributerja, preostanek procesa pa je enak. [7]

4.5 Ubuntu

Ubuntu temelji na Debian pristopu. Razlikujeta se pri pridobivanju novih paketov. Debian je stabilna veja, ki se redko posodablja, Ubuntu pa se konstantno posodablja z najnovejšimi LTS in drugimi verzijami. Ima velike posodobitvene cikle večkrat na leto, med tem ko ma Debian le kakšne varnostne popravke. [31]

4.6 Red Hat Enterprise Linux družina

Paketni format RPM je tu osnova popravkov in posodabljanj. Format `.rpm` je podoben `.deb`. Vsebuje datoteke, skripte, metapodatke, razlikuje pa se v podpisu. RPM paketi

imajo edinstven GPG podpis, ki se preverja na RPM-validatorjih. Nato nadzornik paketov (novejši DNF ali starejši YUM) preveri ali je podpis veljaven ter namestita datoteke iz paketa v `/boot`, nato pa pokličeta skripte, ki pokličejo `dracut`. Ta operacija poskrbi za konfiguracije in nastavi posodobljeno jedro kot osnovno. Prednost te družine pred Debian in Ubuntu je, da zagonilni nalagalnik naloži več jeder v pomnilnik, kar omogoča fallback, v primeru, da se novo jedro neuspešno zažene.

Uporabnik lahko tudi sam naloži RPM pakete, ki so lahko ali od distributerja ali pa lastni, če imajo pravi GPG podpis. [26] [27]

4.7 Fedora “Atomic/Immutable” RPM-OSTREE Model

Ta model, prav tako uporablja RPM pakete, vendar na popolnoma drugačen način. Debian in RHEL posodabljata drevo “v živo”, ko spreminjata `/boot` imenik. Rpm-ostree model, pa je zamišljen nekoliko drugače. “Offline” se pripravi nova celovita slika sistema, na podlagi popravkov iz paketov, torej imamo popolnoma ločen sistem. Nato pa ob ponovnem zagonu nalagalnik naloži novejšo sliko sistema, V primeru, da pa ta ni uspešna, enostavno preklopi nazaj na staro. Celotna ideja je podobna drevesni strukturi git-a. Nadaljevanje je pa podobno kot pri RHEL, z GRUB2 zaganjalnim nalagalnikom.

4.8 MacOS

Pri MacOSu za posodabljanje in krpanje skrbi Softwareupdated daemon. Na začetku preko Software Update ugotovi, če je na voljo kakšen popravek ter, če ima računalnik primerno verzijo in druge pogoje, za nalaganje popravka. Prenos datotek v ozadju opravi softwareupdate_download_service daemon. [2] [29]

Apple ima posebno strukturo jedra, ki je razdeljena na dve particiji. Prva particija je Signed System Volume (SSV), ki hrani sistemske datoteke in jedro je read-only, zapečatena in zavarovana. Iz te particije računalnik pridobi podatke, za zagon in nalaganje osnovnih procesov.

Druga particija je Data Volume, ki hrani uporabniške datoteke, cache, baze ipd in je popolnoma spremenljiva. [2]

Softwareupdated daemon ob prejemu vseh datotek najprej preveri podpis paketov in metapodatke ter nato operacijskemu sistemu pripravi novo particijo, kamor se zgradi popolnoma neodvisen sistem. Na koncu sistem za vsako datoteko izračuna zgoščeno vrednost in jo shrani v "Merkle tree" ter particijo zapečati. Nato v zaganjalnik doda nov zapis, ki kaže na novo sliko. Ob ponovnem zagonu bo nalagalnik naložil novo sliko in s pomočjo Merkle tree preveril njeno veljavnost. V primeru, da pride do napak, lahko naloži prejšnjo sliko sistema, saj SSV ohranja slike in jih aktivno ne briše. [2]

Apple ima tudi posebno tehnologijo za manjše popravke: Rapid Security Response (RSR). Ker je generiranje novih slik težko in počasno, Apple krpanje rešuje drugače. Uvedli so Cryptex, kriptografsko podpisano sliko diska, ki lahko vsebuje dele sistema in se nahaja v preboot volumnu kot ovojnica trenutne slike. Bralnik daje prednost funkcijam v Cryptexu, če pa jih tam ne najde, gre pogledat v SSV. Prednost je, da je za razliko od SSV-ja Cryptex popolnoma spremenljiv ter se lahko doda kot ovoj celo med tekom računalnika. Kljub temu je ponovni zagon računalnika potreben, če se popravek nanaša na širšo komponento sistema. [24]

5. Implementacija preprostega krpalnika v pythonu

Izvirno kodo do enostavnega krpalnika v pythonu, skupaj s skripto in enostavnim projektom za testiranje, lahko najdete na povezavi:

<https://github.com/zangrcar/fri-spo/tree/main/patcher-seminar>

Zaključek

Obstajajo torej različne vrste krpanja, ki se uporabljajo za različne namene. Krpanje izvirne kode predvsem na odprtokodnih projektih, kjer je koda javno dostopna ter pri orodjih za verzioniranje. Za razliko se binarno delta krpanje uporablja v okoljih, kjer izvirna koda ni dostopna, ali kje je hitrost prenosa ključnega pomena. Krpanje med obratovanjem se uporablja predvsem v sistemih, kje ponovni zagon ni sprejemljiv, kot so strežniška okolja, oblačne storitve in jedra operacijskih sistemov.

Literatura

- [1] Ably, Michael. "Delta Compression: Diff Algorithms And Delta File Formats [Practical Guide] | HackerNoon." *Hackernoon*, 22 August 2020,
<https://hackernoon.com/delta-compression-diff-algorithms-and-delta-file-formats-practical-guide-7v1p3uhz>. Accessed 4 January 2026.
- [2] "Apple Platform Deployment – Apple Support (UK)." *Apple Support*,
<https://support.apple.com/en-gb/guide/deployment/welcome/web>. Accessed 4 January 2026.
- [3] ASUS. "[Windows 11/10] Introduction to the Windows Update Feature | Official Support."
ASUS, 17 December 2025, <https://www.asus.com/support/faq/1044671/>. Accessed 3 January 2026.
- [4] Blinn, Bruce. "Patching Source Code." *Bruce Blinn*, 2002,
<https://www.bruceblinn.com/5-LinuxCorner/Patching.html>. Accessed 4 Januar 2026.
- [5] Cruz, Franz da. "IBM Mainframe Magnetic Storage Media." *Columbia University Computing History*, Columbia University, 1 September 2021,
<https://www.columbia.edu/cu/computinghistory/media.html>. Accessed 2 Januar 2021.
- [6] Dadzie, Joseph. "Understanding Software Patching." *ACM Digital Library*, 1 Marec 2005,
<https://spawn-queue.acm.org/doi/full/10.1145/1053331.1053343>. Accessed 28 December 2025.
- [7] Debian Documentation. "update-initramfs." *Debian*, 16 Oktober 2014,
<https://manpages.debian.org/testing/initramfs-tools/update-initramfs.8.en.html>. Accessed 4 Januar 2026.
- [8] Eggert, Paul, et al. "diff(1) - Linux manual page." *man7.org*, Avgust 2025,
<https://man7.org/linux/man-pages/man1/diff.1.html>. Accessed 4 January 2026.

- [9] Farquhar, David L. "Why we call software updates patches." *The Silicon Underground*, 11 Marec 2025, <https://dfarq.homeip.net/why-we-call-software-updates-patches>. Accessed 2 Januar 2026.
- [10] Gala, Jay. "Posix Basic Regular Expressions." *GeeksForGeeks*, 24 Maj 2022, <https://www.geeksforgeeks.org/compiler-design/posix-basic-regular-expressions/>. Accessed 4 Januar 2026.
- [11] Git. "git apply." *git*, August 2025, <https://git-scm.com/docs/git-apply>. Accessed 4 Januar 2026.
- [12] Git. "git-diff Documentation." *Git*, November 2025, <https://git-scm.com/docs/git-diff>. Accessed 4 January 2026.
- [13] Harvard Museums of Science & Culture. "Harvard IBM Mark I - Language | Collection of Historical Scientific Instruments." *Collection of Historical Scientific Instruments*, Harvard University, <https://chsi.harvard.edu/harvard-ibm-mark-1-language>. Accessed 2 January 2026.
- [14] Hunt, J. W., and M. Douglas McIlroy. *An Algorithm for Differential File Comparison*. 1976. *Publications by M. D. McIlroy*, Bell Telephone Laboratories CSTR, <https://www.cs.dartmouth.edu/~doug/diff.pdf>. Accessed 4 Januar 2026.
- [15] The IEEE, and The Open Group. "patch." *The Open Group Base Specifications Issue 6*, 2004, <https://pubs.opengroup.org/onlinepubs/009604499/utilities/patch.html>. Accessed 4 Januar 2026.
- [16] Iyigun, Mehmet. "Hotpatching on Windows." *Microsoft Community Hub*, 19 November 2021, <https://techcommunity.microsoft.com/blog/windowsplatform/hotpatching-on-windows/2959541>. Accessed 4 January 2026.
- [17] Linux. "patch(1)." *Linux manual*, ., <https://man7.org/linux/man-pages/man1/patch.1.html>. Accessed 4 Januar 2026.

- [18] “The Linux Kernel documentation.” *The Linux Kernel documentation — The Linux Kernel documentation*, <https://docs.kernel.org/>. Accessed 4 January 2026.
- [19] Microsoft. “Hotpatch for Windows Server.” *Microsoft Learn*, 17 July 2025, <https://learn.microsoft.com/en-us/windows-server/get-started/hotpatch>. Accessed 4 January 2026.
- [20] Microsoft. “How Windows Update works.” *Microsoft Learn*, 18 January 2023, <https://learn.microsoft.com/en-us/windows/deployment/update/how-windows-update-works>. Accessed 4 January 2026.
- [21] Microsoft. “Moving and Replacing Files.” *Microsoft Learn*, 14 Junij 2022, <https://learn.microsoft.com/en-us/windows/win32/fileio/moving-and-replacing-files>. Accessed 4 Januar 2026.
- [22] Microsoft. “Servicing stack updates.” *Microsoft Learn*, 22 January 2025, <https://learn.microsoft.com/en-us/windows/deployment/update/servicing-stack-updates>. Accessed 4 January 2026.
- [23] 1E Documentation. “Windows 10 Express Installation Files and Delta Content for Updates.” *1E Documentation*, 1 November 2022, <https://help.1e.com/1E23.11/en/803955-804293-windows-10-express-installation-files-and-delta-content-for-updates.html>. Accessed 4 Januar 2026.
- [24] “Quick look at macOS Rapid Security Response – random blog.” *threedots!*, 14 June 2022, <https://threedots.ovh/blog/2022/06/a-quick-look-at-macos-rapid-security-response/>. Accessed 4 January 2026.
- [25] RangeForce Team. “Tutorial: Binary Patching.” *RangeForce*, 20 November 2023, <https://www.rangeforce.com/blog/tutorial-binary-patching>. Accessed 4 January 2026.
- [26] Red Hat Documentation. “Chapter 5. Manually upgrading the kernel | Kernel Administration Guide | Red Hat Enterprise Linux | 7.” *Red Hat Documentation*,

- https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/kernel_administration_guide/ch-manually_upgrading_the_kernel. Accessed 4 January 2026.
- [27] Red Hat, Inc. "Client administration | rpm-ostree." *CoreOS*,
<https://coreos.github.io/rpm-ostree/administrator-handbook/>. Accessed 4 January 2026.
- [28] "Roles and metadata | TUF." *The Update Framework*, 17 April 2025,
<https://theupdateframework.io/docs/metadata/>. Accessed 4 January 2026.
- [29] "softwareupdate_download_service." *manpagez: man pages & more*, 5 November 2017,
https://www.manpagez.com/man/8/softwareupdate_download_service/. Accessed 3
Januar 2026.
- [30] Stewart, Heath. "How Patching Works." *Microsoft*, 12 September 2005,
<https://devblogs.microsoft.com/setup/how-patching-works/>. Accessed 4 Januar 2026.
- [31] Ubuntu. "Automatic updates - Ubuntu Server documentation." *Ubuntu documentation*, 15
December 2015,
<https://documentation.ubuntu.com/server/how-to/software/automatic-updates/>. Accessed
4 January 2026.
- [32] Venkatramana, Shrijith. "Understanding Diff Formats: A Developer's Guide to Making Sense
of Changes." *DEV Community*, 14 Julij 2025,
[https://dev.to/shrsv/understanding-diff-formats-a-developers-guide-to-making-sense-of-c
hanges-414o](https://dev.to/shrsv/understanding-diff-formats-a-developers-guide-to-making-sense-of-changes-414o). Accessed 4 January 2026.